

DSA Tutorial

A Complete Guide

15 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/dsa/>

Getting Started

Welcome to the Data Structures and Algorithms tutorial. This chapter introduces the fundamental concepts you need before diving into specific data structures and algorithms.

What Are Data Structures?

A **data structure** is a way of organizing and storing data so that it can be used efficiently. Common examples include arrays, linked lists, stacks, queues, trees, and graphs. Choosing the right data structure can make your programs faster and easier to maintain.

What Are Algorithms?

An **algorithm** is a step-by-step procedure for solving a problem. Algorithms operate on data structures to perform tasks like searching, sorting, and traversal. The same problem can often be solved by many different algorithms, each with its own trade-offs in speed and memory usage.

Why DSA Matters

Data structures and algorithms are the backbone of computer science. They help you:

- Write code that runs faster and uses less memory
- Tackle complex problems by breaking them into manageable pieces
- Perform well in technical interviews at top companies
- Understand how databases, operating systems, and networks work under the hood

Did you know? Nearly every major software system — from Google Search to Uber's ride matching — relies on efficient data structures and algorithms to serve millions of users in real time.

Simple Algorithm Examples

Finding the Maximum Element

Given an array of numbers, find the largest value. This simple algorithm scans each element once:

```
def find_max(arr):
    max_val = arr[0]
    for val in arr:
        if val > max_val:
            max_val = val
    return max_val
```

Summing Elements

```
def sum_array(arr):
    total = 0
    for val in arr:
        total += val
    return total
```

Measuring Efficiency

We use **Big-O notation** to describe how an algorithm's runtime grows as input size increases. The examples above run in **$O(n)$** time — linear in the number of elements. Throughout this tutorial you will encounter many different complexity classes: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and more.

Key insight: An $O(n)$ algorithm that takes 1 second for 1,000 elements will take roughly 10 seconds for 10,000 elements. In contrast, an $O(n^2)$ algorithm would take about 100 seconds for the same input size — a dramatic difference.

 DSA overview

Topics in This Tutorial

This tutorial covers all the essential topics in a typical DSA curriculum:

- Arrays — static vs dynamic, sorting, and searching
- Linked lists — singly, doubly, and circular
- Stacks and queues — LIFO, FIFO, and their applications
- Hash tables — fast key-value lookups
- Trees — binary trees, BSTs, AVL trees, and heaps
- Graphs — BFS, DFS, cycle detection, and shortest paths
- And more: dynamic programming, greedy algorithms, and complexity analysis

Exercise: Write a function that takes an array of integers and returns both the minimum and maximum values. What is the time complexity of your solution?

Arrays

Arrays are the simplest and most widely used data structure. They store elements in contiguous memory locations and provide $O(1)$ access by index.

Array Basics

An array is a collection of elements of the same type placed in adjacent memory slots. Each element can be accessed directly using its index. The index typically starts at 0.

Memory Layout

If an array starts at memory address `base` and each element takes `size` bytes, then element `i` is located at `base + i * size`. This formula gives constant-time access.

Static vs Dynamic Arrays

Static arrays have a fixed size set at creation. **Dynamic arrays** (like Python's list or Java's ArrayList) automatically grow when elements are added, typically by doubling capacity, which gives amortized $O(1)$ append.

Memory tip: Cache locality makes arrays extremely fast. When you access one element, the CPU loads a whole block of adjacent memory into cache, making subsequent accesses very cheap.

Sorting Algorithms

Sorting arranges elements in a specified order (usually ascending). Here are common sorting algorithms:

Bubble Sort — $O(n^2)$

Repeatedly steps through the list, swaps adjacent elements if they are in the wrong order, and repeats until no swaps are needed.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Selection Sort — $O(n^2)$

Finds the smallest element and swaps it with the first unsorted position.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Insertion Sort — $O(n^2)$

Builds the sorted array one element at a time by inserting each element into its correct position.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

Merge Sort — $O(n \log n)$

A divide-and-conquer algorithm that splits the array in half, recursively sorts each half, then merges the sorted halves.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]
```

Quick Sort — $O(n \log n)$ average

Selects a pivot, partitions the array into elements less than and greater than the pivot, then recursively sorts each partition.

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

Counting Sort — $O(n + k)$

A non-comparison sort that counts occurrences of each distinct value. Works well when the range of possible values (k) is small.

```
def counting_sort(arr):
    max_val = max(arr)
    count = [0] * (max_val + 1)
    for val in arr:
        count[val] += 1
    sorted_arr = []
    for i in range(len(count)):
        sorted_arr.extend([i] * count[i])
    return sorted_arr
```



Array memory representation

Searching Algorithms

Linear Search — $O(n)$

Scans every element until the target is found. Simple but slow for large arrays.

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
```

Binary Search — $O(\log n)$

Works on **sorted** arrays. Repeatedly divides the search interval in half.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Key insight: Binary search is exponentially faster than linear search on large sorted arrays. For an array of 1 million elements, linear search takes up to 1 million steps, while binary search takes at most 20.

Time Complexity Summary

- Access by index: $O(1)$
- Search (unsorted, linear): $O(n)$
- Search (sorted, binary): $O(\log n)$
- Insertion at end (dynamic array, amortized): $O(1)$
- Insertion at beginning/middle: $O(n)$
- Deletion: $O(n)$

Exercise: Implement a function that rotates an array to the right by k steps. For example, `rotate([1, 2, 3, 4, 5], 2)` should return `[4, 5, 1, 2, 3]`. Try to solve it in-place with $O(1)$ extra space.

Linked Lists

A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node. Unlike arrays, linked lists do not require contiguous memory and can grow or shrink dynamically.

Node Structure

Each node contains two fields: the **data** and a **pointer** (or reference) to the next node. The list is accessed through a **head** pointer that points to the first node.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Singly Linked List

In a singly linked list, each node has a single pointer to the next node. The last node points to `None`.

Insert at Head

```
def insert_at_head(head, data):
    new_node = Node(data)
    new_node.next = head
    return new_node
```

Insert at Tail

```
def insert_at_tail(head, data):
    if head is None:
        return Node(data)
    current = head
    while current.next:
        current = current.next
    current.next = Node(data)
    return head
```

Traverse

```
def traverse(head):
    current = head
    while current:
        print(current.data, end=" -> ")
        current = current.next
    print("None")
```

Search

```
def search(head, target):
    current = head
    while current:
        if current.data == target:
            return True
        current = current.next
    return False
```

Delete a Node

```
def delete_node(head, key):
    if head is None:
        return None
    if head.data == key:
        return head.next
    current = head
    while current.next and current.next.data != key:
        current = current.next
    if current.next:
        current.next = current.next.next
    return head
```

Reverse

```
def reverse_list(head):
    prev = None
    current = head
    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node
    return prev
```

 Singly linked list diagram

Doubly Linked List

Each node has two pointers: `prev` (to the previous node) and `next` (to the next node). This allows traversal in both directions and makes deletion at the tail $O(1)$ when a tail pointer is maintained.

```
class DoublyNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Circular Linked List

The last node points back to the head, forming a circle. Useful for round-robin scheduling and applications that cycle through a list repeatedly.

Use case: Circular linked lists are used in operating systems for CPU scheduling (round-robin), in multiplayer games for turn management, and in music playlists that loop.

Linked List vs Arrays

- **Memory:** Linked lists use non-contiguous memory; arrays use contiguous memory.
- **Access:** Arrays provide $O(1)$ random access; linked lists require $O(n)$ traversal.
- **Insertion/Deletion:** Linked lists offer $O(1)$ at head/tail (given a pointer); arrays are $O(n)$ for insertions and deletions except at the end.
- **Cache performance:** Arrays benefit from spatial locality; linked lists suffer from cache misses due to scattered nodes.
- **Size:** Linked lists grow naturally; static arrays have a fixed size.

Key insight: Linked lists shine when you need frequent insertions and deletions at arbitrary positions. Arrays shine when you need fast random access and iteration with good cache locality.

Time Complexities

- Access: $O(n)$

- Search: $O(n)$
- Insert at head: $O(1)$
- Insert at tail (with tail pointer): $O(1)$
- Delete at head: $O(1)$
- Delete by value: $O(n)$

Exercise: Implement a function that detects if a linked list has a cycle (i.e., a node's `next` pointer eventually points back to a previous node). Use Floyd's tortoise-and-hare algorithm (two pointers moving at different speeds).

Stacks & Queues

Stacks and queues are abstract data types that restrict how elements are added and removed. They model real-world waiting lines and last-in-first-out behavior.

Stack — LIFO (Last In, First Out)

A stack is like a stack of plates: you add to the top and remove from the top. The last item placed is the first one retrieved.

Stack Operations

- `push(item)` — adds an item to the top
- `pop()` — removes and returns the top item
- `peek()` — returns the top item without removing it
- `is_empty()` — checks if the stack is empty

Array-Based Stack

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if self.is_empty():
            return None
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            return None
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0
```

Linked List Stack

```
class StackNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedListStack:
    def __init__(self):
        self.top = None

    def push(self, data):
        node = StackNode(data)
        node.next = self.top
        self.top = node

    def pop(self):
        if self.top is None:
            return None
        data = self.top.data
        self.top = self.top.next
        return data

    def peek(self):
        return self.top.data if self.top else None

    def is_empty(self):
        return self.top is None
```

Stack Applications

- **Undo/Redo:** Text editors push each change onto a stack; Undo pops it.
- **Call Stack:** Functions push a stack frame when called and pop it on return.
- **Expression Evaluation:** Converting infix to postfix, evaluating postfix expressions.
- **Syntax Parsing:** Checking matching parentheses, braces, and brackets.

Queue — FIFO (First In, First Out)

A queue is like a line at a ticket counter: the first person in line is the first one served.

Queue Operations

- `enqueue(item)` — adds an item to the rear
- `dequeue()` — removes and returns the front item
- `front()` — returns the front item without removing it
- `is_empty()` — checks if the queue is empty

Array-Based Queue

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            return None
        return self.items.pop(0)

    def front(self):
        return self.items[0] if not self.is_empty() else None

    def is_empty(self):
        return len(self.items) == 0
```

Circular Queue

A circular queue reuses array slots by wrapping around. It uses a fixed-size array with front and rear pointers.

```

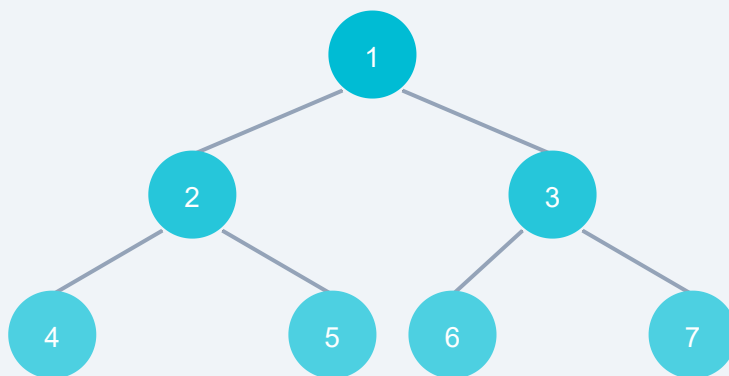
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.items = [None] * capacity
        self.front = 0
        self.rear = 0
        self.size = 0

    def enqueue(self, item):
        if self.size == self.capacity:
            return False
        self.items[self.rear] = item
        self.rear = (self.rear + 1) % self.capacity
        self.size += 1
        return True

    def dequeue(self):
        if self.size == 0:
            return None
        item = self.items[self.front]
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return item

```

Binary Tree Traversal



Deque (Double-Ended Queue)

Allows insertion and deletion from both ends. Python's `collections.deque` is a highly optimized implementation.

Priority Queue

Elements are dequeued in order of priority, not insertion order. Typically implemented with a binary heap. Operations: $O(\log n)$ for insertion and deletion.

Key insight: Breadth-First Search (BFS) uses a queue to explore nodes level by level. The undo feature in every editor is a stack. These structures appear everywhere in software engineering.

Time Complexities

- Stack push/pop/peek: $O(1)$
- Queue enqueue/dequeue: $O(1)$ amortized (or with circular buffer)
- Priority queue insert/delete: $O(\log n)$

Exercise: Implement a function that uses a stack to check if a string of parentheses, braces, and brackets is balanced. For example, `"{[{()]}"` is balanced but `"{([})]"` is not.

Hash Tables

Hash tables provide lightning-fast key-value lookups with $O(1)$ average time complexity. They are the engine behind dictionaries, maps, and associative arrays in almost every programming language.

Hash Table Concept

A hash table stores data in key-value pairs. A **hash function** takes a key and computes an index (a hash code) that determines where the value is stored in an internal array. When you look up a key, the same hash function is used to find its location directly.

Hash Functions

A good hash function should be:

- **Deterministic:** same key always produces the same hash code
- **Fast to compute**
- **Uniformly distributed:** minimizes collisions by spreading keys evenly
- **Non-invertible:** hard to reconstruct the key from its hash

```
def simple_hash(key, table_size):  
    # Simple hash for string keys  
    total = 0  
    for ch in str(key):  
        total += ord(ch)  
    return total % table_size
```

Real-world hash functions: Python uses SipHash for string keys; Java uses a polynomial hash; cryptographic hashes like SHA-256 are used for security.

Collision Resolution

Collisions happen when two different keys hash to the same index. Here are the main strategies:

1. Chaining (Separate Chaining)

Each bucket stores a linked list (or another collection) of key-value pairs. When a collision occurs, the new pair is appended to the list at that index.

```
class HashTableChaining:
    def __init__(self, capacity=10):
        self.capacity = capacity
        self.table = [[] for _ in range(capacity)]

    def _hash(self, key):
        return hash(key) % self.capacity

    def put(self, key, value):
        idx = self._hash(key)
        for i, (k, v) in enumerate(self.table[idx]):
            if k == key:
                self.table[idx][i] = (key, value)
                return
        self.table[idx].append((key, value))

    def get(self, key):
        idx = self._hash(key)
        for k, v in self.table[idx]:
            if k == key:
                return v
        return None

    def remove(self, key):
        idx = self._hash(key)
        self.table[idx] = [(k, v) for k, v in self.table[idx] if k != key]
```

2. Open Addressing

When a collision occurs, probe for the next empty slot. Variants include:

- **Linear probing:** check slot + 1, + 2, + 3, ...
- **Quadratic probing:** check slot + 1², + 2², + 3², ...
- **Double hashing:** use a second hash function to determine the step size

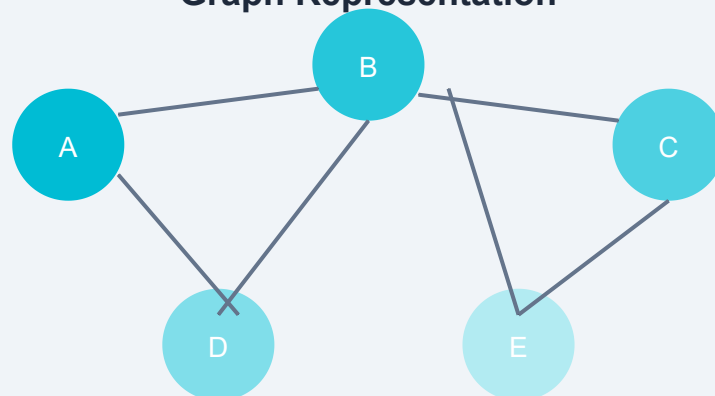
```
class HashTableLinearProbing:
    def __init__(self, capacity=10):
        self.capacity = capacity
        self.keys = [None] * capacity
        self.values = [None] * capacity

    def _hash(self, key):
        return hash(key) % self.capacity

    def put(self, key, value):
        idx = self._hash(key)
        while self.keys[idx] is not None:
            if self.keys[idx] == key:
                self.values[idx] = value
                return
            idx = (idx + 1) % self.capacity
        self.keys[idx] = key
        self.values[idx] = value

    def get(self, key):
        idx = self._hash(key)
        while self.keys[idx] is not None:
            if self.keys[idx] == key:
                return self.values[idx]
            idx = (idx + 1) % self.capacity
        return None
```

Graph Representation



Load Factor and Resizing

The **load factor** is the ratio of stored entries to table capacity. When it exceeds a threshold (typically 0.75), the table is resized (usually doubled in capacity) and all entries are rehashed. This keeps operations fast at the cost of occasional $O(n)$ resizing.

Hash Set vs Hash Map

- **Hash Set:** stores only keys (no values). Used to track membership (e.g., visited nodes, unique elements).
- **Hash Map:** stores key-value pairs. Used for lookups, caching, counting frequencies, and associative data.

Key insight: Hash tables give you $O(1)$ average performance for insert, delete, and lookup. This makes them the go-to choice whenever you need to associate values with unique keys or quickly check membership.

Applications

- Database indexing
- Cache systems (memcached, Redis)
- Compilers and interpreters (symbol tables)
- Counting word frequencies
- Detecting duplicates
- Two-sum problem and other interview classics

Exercise: Given an array of integers, find two numbers that add up to a target sum. Implement a solution using a hash map that runs in $O(n)$ time. For example, `two_sum([2, 7, 11, 15], 9)` should return `[0, 1]`.

Trees

Trees are hierarchical data structures. Unlike arrays and linked lists which are linear, trees branch into parent-child relationships. They are used everywhere: file systems, HTML DOM, database indexes, and network routing.

Tree Terminology

- **Root:** the topmost node with no parent
- **Child:** a node directly connected below another node
- **Parent:** a node directly connected above a child
- **Leaf:** a node with no children
- **Depth:** number of edges from the root to the node
- **Height:** number of edges on the longest path from the node to a leaf

Binary Trees

A **binary tree** is a tree where each node has at most two children, called **left** and **right**.

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

Tree Traversals

Preorder (Root → Left → Right)

```
def preorder(node):  
    if node:  
        print(node.val, end=" ")  
        preorder(node.left)  
        preorder(node.right)
```

Inorder (Left → Root → Right)

```
def inorder(node):  
    if node:  
        inorder(node.left)  
        print(node.val, end=" ")  
        inorder(node.right)
```

Postorder (Left → Right → Root)

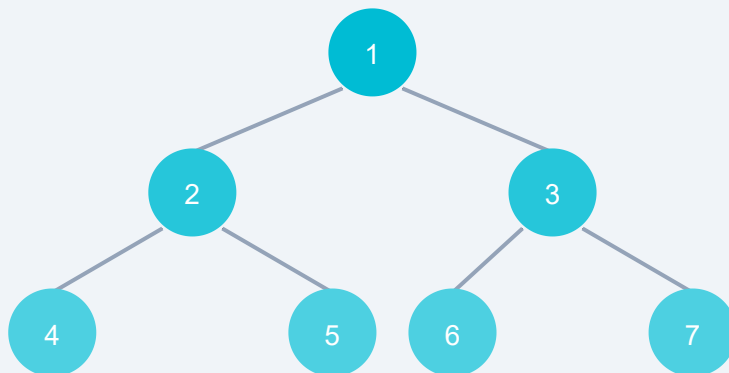
```
def postorder(node):  
    if node:  
        postorder(node.left)  
        postorder(node.right)  
        print(node.val, end=" ")
```

Level-Order (BFS)

```
from collections import deque

def level_order(root):
    if not root:
        return
    q = deque([root])
    while q:
        node = q.popleft()
        print(node.val, end=" ")
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
```

Binary Tree Traversal



Array Representation of Binary Tree

A binary tree can be stored in an array: index 0 holds the root, and for any node at index i , its left child is at $2i + 1$ and right child at $2i + 2$. This is the basis for heap data structures.

Binary Search Tree (BST)

A BST is a binary tree where for every node: all values in the left subtree are smaller, and all values in the right subtree are larger.

Insert

```
def insert(root, val):
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root
```

Search

```
def search(root, target):
    if root is None or root.val == target:
        return root
    if target < root.val:
        return search(root.left, target)
    return search(root.right, target)
```

Delete

```
def delete(root, val):
    if root is None:
        return None
    if val < root.val:
        root.left = delete(root.left, val)
    elif val > root.val:
        root.right = delete(root.right, val)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        # Node with two children: find inorder successor
        succ = root.right
        while succ.left:
            succ = succ.left
        root.val = succ.val
        root.right = delete(root.right, succ.val)
    return root
```

AVL Trees (Balanced BST)

AVL trees maintain balance using a **balance factor** (height of left - height of right). When a node becomes unbalanced (factor ± 2), one of four rotations restores balance: left, right, left-right, or right-left rotation.

Why balance matters: A BST can degenerate into a linked list in the worst case ($O(n)$ operations). AVL trees guarantee $O(\log n)$ for insert, delete, and search by keeping the tree height within $\log n$.

Heap (Binary Heap)

A heap is a complete binary tree that satisfies the **heap property**:

- **Max-heap:** each parent is greater than or equal to its children
- **Min-heap:** each parent is less than or equal to its children

Heapify (Building a heap from an array)

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def build_max_heap(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
```

Key insight: Heaps are the classic implementation of priority queues. Heapsort uses a max-heap to sort an array in $O(n \log n)$ time with $O(1)$ extra space.

Exercise: Implement a function that checks if a binary tree is a valid BST. For every node, all values in its left subtree must be less, and all values in its right subtree must be greater. Use an in-order traversal approach.

Graphs

Graphs are the most flexible and expressive data structure. They model relationships between entities — think social networks, road maps, web links, and dependency graphs.

Graph Terminology

- **Vertex (node):** a fundamental unit of the graph
- **Edge:** a connection between two vertices
- **Directed graph (digraph):** edges have a direction ($A \rightarrow B$)
- **Undirected graph:** edges are bidirectional ($A — B$)
- **Weighted graph:** edges carry a numerical weight (cost, distance, etc.)
- **Path:** a sequence of vertices connected by edges
- **Cycle:** a path that starts and ends at the same vertex

Graph Representations

Adjacency Matrix

A 2D array where `matrix[i][j]` is 1 (or the edge weight) if there is an edge from vertex i to vertex j . Space: $O(V^2)$. Good for dense graphs.

```

class GraphMatrix:
    def __init__(self, num_vertices):
        self.n = num_vertices
        self.matrix = [[0] * num_vertices for _ in range(num_vertices)]

    def add_edge(self, u, v, weight=1):
        self.matrix[u][v] = weight
        # For undirected:
        # self.matrix[v][u] = weight

    def has_edge(self, u, v):
        return self.matrix[u][v] != 0

```

Adjacency List

An array of lists where each vertex stores a list of its neighbors. Space: $O(V + E)$.
Good for sparse graphs.

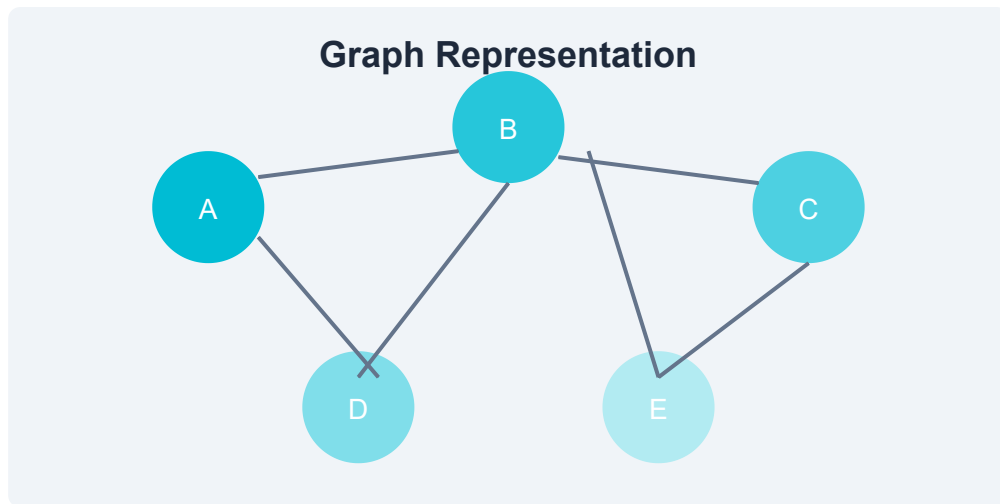
```

class GraphList:
    def __init__(self, num_vertices):
        self.n = num_vertices
        self.adj = [[] for _ in range(num_vertices)]

    def add_edge(self, u, v, weight=1):
        self.adj[u].append((v, weight))
        # For undirected:
        # self.adj[v].append((u, weight))

    def get_neighbors(self, u):
        return self.adj[u]

```



Breadth-First Search (BFS)

Explores the graph level by level. Uses a queue. Finds the shortest path in unweighted graphs.

```
from collections import deque

def bfs(graph, start):
    visited = [False] * graph.n
    q = deque([start])
    visited[start] = True
    while q:
        v = q.popleft()
        print(v, end=" ")
        for neighbor, _ in graph.adj[v]:
            if not visited[neighbor]:
                visited[neighbor] = True
                q.append(neighbor)
```

Depth-First Search (DFS)

Recursive DFS

```
def dfs_recursive(graph, v, visited):
    visited[v] = True
    print(v, end=" ")
    for neighbor, _ in graph.adj[v]:
        if not visited[neighbor]:
            dfs_recursive(graph, neighbor, visited)
```

Iterative DFS (using a stack)

```
def dfs_iterative(graph, start):
    visited = [False] * graph.n
    stack = [start]
    while stack:
        v = stack.pop()
        if not visited[v]:
            visited[v] = True
            print(v, end=" ")
            for neighbor, _ in reversed(graph.adj[v]):
                if not visited[neighbor]:
                    stack.append(neighbor)
```

Cycle Detection

For undirected graphs, use DFS and check if a neighbor is visited and is not the parent. For directed graphs, use a recursion stack to track nodes currently in the recursion path.

```

def has_cycle_directed(graph):
    visited = [False] * graph.n
    rec_stack = [False] * graph.n

    def dfs(v):
        visited[v] = True
        rec_stack[v] = True
        for neighbor, _ in graph.adj[v]:
            if not visited[neighbor]:
                if dfs(neighbor):
                    return True
            elif rec_stack[neighbor]:
                return True
        rec_stack[v] = False
        return False

    for v in range(graph.n):
        if not visited[v]:
            if dfs(v):
                return True
    return False

```

Topological Sort (DAG)

A topological ordering of a Directed Acyclic Graph (DAG) arranges vertices such that for every edge $u \rightarrow v$, u comes before v . Used for task scheduling, build systems, and dependency resolution.

```
def topological_sort(graph):
    visited = [False] * graph.n
    stack = []

    def dfs(v):
        visited[v] = True
        for neighbor, _ in graph.adj[v]:
            if not visited[neighbor]:
                dfs(neighbor)
        stack.append(v)

    for v in range(graph.n):
        if not visited[v]:
            dfs(v)

    return stack[::-1]
```

Applications: Google Maps uses graph algorithms for route finding. Facebook's friend suggestions rely on graph traversal. Package managers use topological sort to resolve dependencies.

Key insight: BFS finds the shortest path in unweighted graphs. DFS uses less memory on average and is good for exploring all paths, detecting cycles, and topological sorting.

Exercise: Implement a function that checks whether an undirected graph is connected (i.e., there is a path between any two vertices). Use BFS or DFS starting from vertex 0 and check if all vertices were visited.

Shortest Path Algorithms

Shortest path algorithms find the minimum distance (or cost) between vertices in a weighted graph. They are fundamental to GPS navigation, network routing, and game AI pathfinding.

Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a single source vertex to all other vertices in a graph with **non-negative** edge weights. It uses a greedy approach: always expand the closest unvisited vertex.

```
import heapq

def dijkstra(graph, start):
    INF = float('inf')
    dist = [INF] * graph.n
    dist[start] = 0
    pq = [(0, start)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph.adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

How it works

1. Initialize distances: source = 0, all others = infinity
2. Push the source into a min-heap priority queue
3. Pop the vertex with the smallest distance

4. For each neighbor, if a shorter path is found, update its distance and push it onto the heap
5. Repeat until the heap is empty

Complexity: $O((V + E) \log V)$ with a binary heap priority queue. Dijkstra's algorithm fails on graphs with negative edge weights because a later edge could reduce a distance that was already finalized.

Bellman-Ford Algorithm

Bellman-Ford solves the single-source shortest path problem even when edges have **negative weights**. It can also detect **negative cycles** (cycles whose total weight is negative).

```
def bellman_ford(graph, start):
    INF = float('inf')
    dist = [INF] * graph.n
    dist[start] = 0

    # Relax all edges V-1 times
    for _ in range(graph.n - 1):
        for u in range(graph.n):
            for v, w in graph.adj[u]:
                if dist[u] != INF and dist[u] + w < dist[v]:
                    dist[v] = dist[u] + w

    # Check for negative cycles
    for u in range(graph.n):
        for v, w in graph.adj[u]:
            if dist[u] != INF and dist[u] + w < dist[v]:
                return None # Negative cycle detected
    return dist
```

Why $V - 1$ iterations?

In a graph with V vertices, the shortest path between any two vertices can have at most $V - 1$ edges (a simple path without cycles). Each iteration relaxes all

edges, and after k iterations, the algorithm has found all shortest paths with at most k edges.

Complexity: $O(V \times E)$. Slower than Dijkstra, but handles negative weights. If a distance still improves on the V -th iteration, a negative cycle exists.

Floyd-Warshall Algorithm

Floyd-Warshall finds the **shortest paths between all pairs** of vertices in $O(V^3)$ time. It works with negative weights (but not negative cycles) and is implemented with dynamic programming.

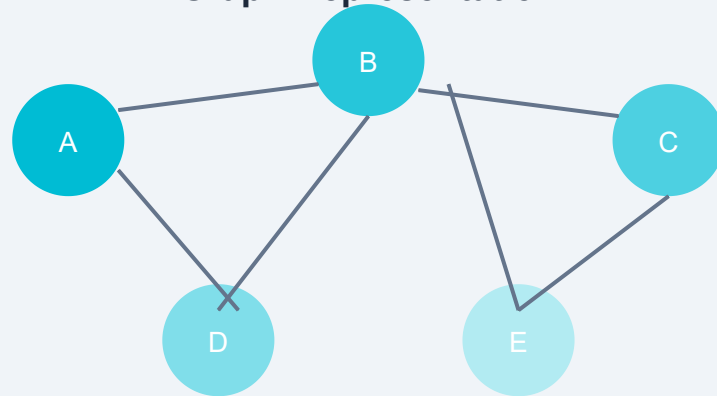
```
def floyd_warshall(graph):
    INF = float('inf')
    n = graph.n
    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0
        for j, w in graph.adj[i]:
            dist[i][j] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist
```

Graph Representation



A* Search Algorithm

A* is a **heuristic-based** shortest path algorithm widely used in games and robotics. It combines Dijkstra's cost from the start with a heuristic estimate of the cost to the goal (e.g., Euclidean or Manhattan distance).

```
import heapq

def a_star(graph, start, goal, heuristic):
    open_set = [(0, start)]
    g_score = {start: 0}
    f_score = {start: heuristic(start, goal)}

    while open_set:
        _, current = heapq.heappop(open_set)
        if current == goal:
            return g_score[current]

        for neighbor, weight in graph.adj[current]:
            tentative_g = g_score[current] + weight
            if tentative_g < g_score.get(neighbor, float('inf')):
                g_score[neighbor] = tentative_g
                f_score[neighbor] = tentative_g + heuristic(neighbor, goal)
                heapq.heappush(open_set, (f_score[neighbor], neighbor))

    return None
```

When to use which?

- **Dijkstra:** non-negative weights, single source, guarantees optimality

- **Bellman-Ford:** handles negative weights, detects negative cycles
- **Floyd-Warshall:** all-pairs shortest paths, dense graphs
- **A*:** fast point-to-point search with a good heuristic

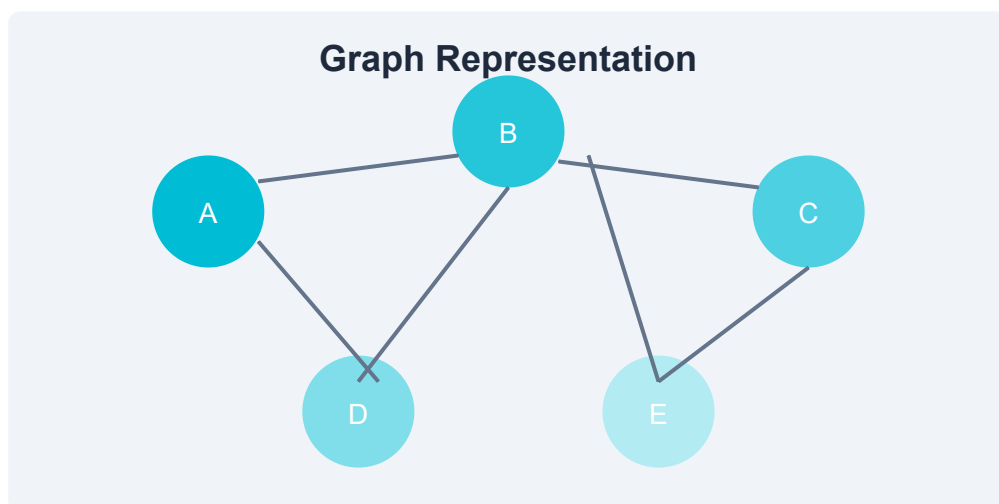
Key insight: A* is essentially Dijkstra with a compass. The heuristic guides the search toward the goal, potentially exploring far fewer vertices. If the heuristic is admissible (never overestimates), A* is guaranteed to find the shortest path.

Exercise: Implement Dijkstra's algorithm and return not just the shortest distances but also the actual path (sequence of vertices) from the source to a specific target. Use a `prev` array to track the predecessor of each vertex.

Spanning Tree & Minimum Spanning Tree

A **spanning tree** of an undirected graph is a subgraph that includes all vertices but only enough edges to keep the graph connected (exactly $|V| - 1$ edges). When edges have weights, the **Minimum Spanning Tree (MST)** is the spanning tree with the smallest total edge weight.

MSTs are the backbone of network design — building roads, laying cables, or connecting computer networks at the lowest possible cost.



Properties of an MST

- Contains exactly $v - 1$ edges for v vertices.
- No cycles — it is a tree by definition.
- The total weight is minimal among all possible spanning trees.
- If all edge weights are distinct, the MST is unique.

Prim's Algorithm (Greedy, Vertex-Based)

Prim's algorithm grows the MST one vertex at a time. It starts from an arbitrary vertex and repeatedly adds the cheapest edge that connects a visited vertex to an unvisited one.

How it works

1. Pick any starting vertex. Mark it as visited.
2. Consider all edges from visited vertices to unvisited vertices.
3. Pick the edge with the smallest weight. Add it to the MST.
4. Mark the newly connected vertex as visited.
5. Repeat steps 2–4 until all vertices are visited.

```
def prim(graph, start):
    n = len(graph)
    visited = [False] * n
    min_edge = [float('inf')] * n
    min_edge[start] = 0
    total_weight = 0

    for _ in range(n):
        u = -1
        # Pick the unvisited vertex with smallest edge
        for v in range(n):
            if not visited[v] and (u == -1 or min_edge[v] < min_edge[u]):
                u = v
        visited[u] = True
        total_weight += min_edge[u]
        # Update edges to neighbors
        for v, w in enumerate(graph[u]):
            if w and not visited[v] and w < min_edge[v]:
                min_edge[v] = w
    return total_weight
```

Time complexity: $O(V^2)$ with an adjacency matrix, or $O(E \log V)$ with a binary heap.

Kruskal's Algorithm (Edge-Based, Union-Find DSU)

Kruskal's algorithm sorts all edges by weight, then adds them one by one — skipping any edge that would create a cycle. It uses a **Disjoint Set Union (DSU)** or **Union-Find** data structure to detect cycles efficiently.

How it works

1. Sort all edges by weight in ascending order.
2. Initialize a DSU where each vertex is its own parent.
3. For each edge (u, v) in sorted order:
 - If `find(u) != find(v)`, add the edge to the MST and union the sets of u and v .
4. Stop when the MST has `v - 1` edges.

```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            self.parent[rx] = ry
        elif self.rank[rx] > self.rank[ry]:
            self.parent[ry] = rx
        else:
            self.parent[ry] = rx
            self.rank[rx] += 1
        return True

    def kruskal(vertices, edges):
        edges.sort(key=lambda e: e[2])
        dsu = DSU(vertices)
        mst_weight = 0
        count = 0
        for u, v, w in edges:
            if dsu.union(u, v):
                mst_weight += w
                count += 1
            if count == vertices - 1:
                break
        return mst_weight

```

Time complexity: $O(E \log E)$ due to sorting, or $O(E \log V)$ with efficient DSU operations.

Comparison: Prim vs Kruskal

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Vertex-based (grows like BFS)	Edge-based (processes sorted edges)
Data Structure	Priority queue / heap	Union-Find (DSU)
Best for	Dense graphs (many edges)	Sparse graphs (few edges)
Time Complexity	$O(E \log V)$ with heap	$O(E \log E)$ — dominated by sorting
Graph Requirement	Connected graph	Connected or disconnected (finds forest)

Both algorithms are greedy and yield the same MST for a given graph. Choose Prim for dense graphs and Kruskal when edges are naturally sorted or when the graph is sparse.

Applications of MST

- **Network design** — laying fiber-optic cables, electrical grids, pipelines.
- **Approximation algorithms** — the MST is used as a building block for approximate solutions to NP-hard problems like the Traveling Salesman Problem.
- **Clustering** — single-linkage clustering is essentially Kruskal's algorithm stopped early.
- **Image segmentation** — graph-based segmentation uses MST concepts.
- **Broadcast routing** — MST minimizes total cost of broadcasting in a network.

Practice Task

Implement both Prim's and Kruskal's algorithms on the following graph (vertices 0–4):

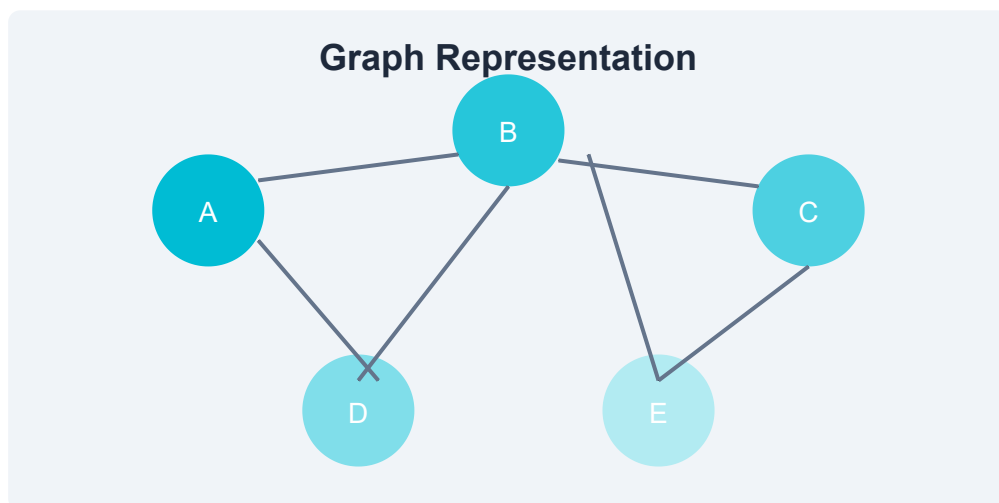
```
edges = [  
    (0, 1, 10), (0, 2, 6), (0, 3, 5),  
    (1, 3, 15), (2, 3, 4), (3, 4, 8)  
]
```

Verify that both algorithms produce the same MST weight. Then modify your code to also return the list of edges in the MST.

Maximum Flow in Networks

A **flow network** is a directed graph where each edge has a **capacity** (the maximum amount of flow it can carry). The goal is to find the maximum amount of flow that can be sent from a **source** vertex (s) to a **sink** vertex (t) while respecting all capacity constraints.

Flow networks model real-world problems like water pipelines, data traffic in computer networks, airline scheduling, and even bipartite matching.



Flow Network Terminology

- **Source (s)** — the vertex where flow originates; has no incoming edges carrying flow.
- **Sink (t)** — the vertex where flow terminates; has no outgoing edges carrying flow.
- **Capacity** — a non-negative integer or real on each edge, denoting the maximum flow through that edge.
- **Flow** — a non-negative value on each edge satisfying $0 \leq \text{flow} \leq \text{capacity}$.
- **Conservation** — for every vertex except s and t , flow in = flow out.

- **Value of flow** — the total flow leaving the source (which equals total flow arriving at the sink).

Ford-Fulkerson Method

The Ford-Fulkerson method is a greedy iterative algorithm. It repeatedly finds an **augmenting path** from s to t in the **residual graph** and pushes flow along that path. The process repeats until no more augmenting paths exist.

Residual Graph

The residual graph G_f shows the remaining capacity on each edge after some flow has been sent. For each original edge (u, v) with capacity c and flow f , the residual graph contains:

- A forward edge (u, v) with residual capacity $c - f$.
- A backward edge (v, u) with residual capacity f (allowing flow to be undone).

```

def ford_fulkerson(graph, source, sink):
    n = len(graph)
    # Residual graph: res[u][v] = remaining capacity
    res = [row[:] for row in graph]
    max_flow = 0

    def bfs(s, t, parent):
        visited = [False] * n
        queue = [s]
        visited[s] = True
        while queue:
            u = queue.pop(0)
            for v in range(n):
                if not visited[v] and res[u][v] > 0:
                    parent[v] = u
                    visited[v] = True
                    queue.append(v)
                    if v == t:
                        return True
            return False

    parent = [-1] * n
    while bfs(source, sink, parent):
        # Find the bottleneck capacity along the augmenting path
        bottleneck = float('inf')
        v = sink
        while v != source:
            u = parent[v]
            bottleneck = min(bottleneck, res[u][v])
            v = u
        # Update residual capacities
        v = sink
        while v != source:
            u = parent[v]
            res[u][v] -= bottleneck
            res[v][u] += bottleneck
            v = u
        max_flow += bottleneck
    return max_flow

```

Edmonds-Karp Algorithm

Edmonds-Karp is a specific implementation of Ford-Fulkerson that uses **BFS** to find the shortest augmenting path (in terms of number of edges). This guarantees $O(V * E^2)$ time complexity, whereas a naive DFS-based Ford-Fulkerson can be exponential.

The key insight: BFS finds the path with the fewest edges, which bounds the number of augmentations to $O(V * E)$. Each BFS costs $O(E)$, yielding $O(V * E^2)$ overall.

Max-Flow Min-Cut Theorem

One of the most important results in network flow theory:

The value of the maximum flow from s to t equals the capacity of the minimum s - t cut.

An **s - t cut** is a partition of vertices into two sets S and T (with s in S and t in T such that the sum of capacities of edges from S to T is minimized). The minimum cut is the bottleneck of the network.

In the residual graph after max flow is computed, the set of vertices reachable from s forms the minimum cut. Edges crossing from S (reachable) to T (unreachable) are the bottleneck edges.

Applications of Maximum Flow

- **Bipartite Matching** — add a source connected to left vertices, connect right vertices to a sink, set all capacities to 1. The max flow equals the size of the maximum matching.
- **Transportation** — shipping goods from factories (sources) to warehouses (sinks) with road capacity limits.

- **Image Segmentation** — treat pixels as vertices; the min cut separates foreground from background.
- **Circulation with Demands** — supply and demand nodes connected by capacity-limited edges.

```
# Bipartite matching via max flow
def max_bipartite_matching(left, right, edges):
    # Build flow network:
    # source = 0, left = 1..|left|, right = |left|+1..|left|+|right|, sink =
    n = 2 + len(left) + len(right)
    graph = [[0]*n for _ in range(n)]
    source, sink = 0, n - 1
    for i in range(len(left)):
        graph[source][1 + i] = 1
    for i in range(len(right)):
        graph[1 + len(left) + i][sink] = 1
    for u, v in edges:
        graph[1 + u][1 + len(left) + v] = 1
    return ford_fulkerson(graph, source, sink)
```

Complexity Summary

Algorithm	Time Complexity	Notes
Ford-Fulkerson (DFS)	$O(E * f)$	Depends on max flow value; can be slow
Edmonds-Karp (BFS)	$O(V * E^2)$	Polynomial; guarantees termination
Dinic's	$O(V^2 * E)$	Faster for unit-capacity graphs; $O(E * \sqrt{V})$ for bipartite matching

Practice Task

Given a flow network with 5 nodes (source=0, sink=4) and the following capacities:

```
graph = [  
    [0, 16, 13, 0, 0],  
    [0, 0, 10, 12, 0],  
    [0, 4, 0, 0, 14],  
    [0, 0, 9, 0, 20],  
    [0, 0, 0, 0, 0]  
]
```

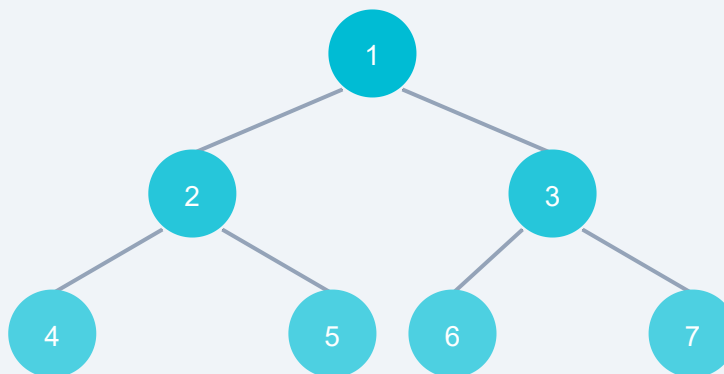
Implement Edmonds-Karp to compute the max flow. Then find the min-cut edges by running BFS on the residual graph.

Time Complexity & Asymptotic Analysis

Time complexity measures how the runtime of an algorithm grows as the input size increases. We use asymptotic notation to express this growth in a hardware-independent way. Understanding complexity lets you choose the right algorithm for large datasets.

An algorithm that runs in $O(n)$ on a dataset of 1 million items takes about 1 million operations. One that runs in $O(n^2)$ takes about 1 trillion — that is the difference between one second and 12 days.

Binary Tree Traversal



Big-O Notation (Upper Bound)

Big-O describes the worst-case growth rate. $f(n) = O(g(n))$ means that for sufficiently large n , $f(n)$ is bounded above by a constant multiple of $g(n)$.

Common Big-O Classes

Notation	Name	Example	Feasible for n
$O(1)$	Constant	Array access, hash lookup	Any size
$O(\log n)$	Logarithmic	Binary search, balanced BST	10^9+
$O(n)$	Linear	Linear search, array traversal	$\sim 10^7$
$O(n \log n)$	Linearithmic	Mergesort, Heapsort	$\sim 10^6$
$O(n^2)$	Quadratic	Bubble sort, nested loops	$\sim 10^4$
$O(2^n)$	Exponential	Fibonacci recursion without memo	~ 30
$O(n!)$	Factorial	Brute-force TSP	~ 12

```

# O(1) – constant time
def get_first(arr):
    return arr[0]

# O(n) – linear time
def linear_search(arr, target):
    for i, v in enumerate(arr):
        if v == target:
            return i
    return -1

# O(n^2) – quadratic time
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# O(log n) – logarithmic time
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

```

Omega Notation & Theta Notation

- **Ω (Omega)** — Lower bound. $f(n) = \Omega(g(n))$ means $f(n)$ grows at least as fast as $g(n)$. Describes the best-case scenario.
- **Θ (Theta)** — Tight bound. $f(n) = \Theta(g(n))$ means $f(n)$ grows exactly as fast as $g(n)$ (both upper and lower bound match). This is the most precise characterization.

Big-O is the most commonly used in practice. However, when analyzing algorithms like QuickSort, you will see $O(n^2)$ worst-case and $\Omega(n \log n)$ best-case — but the expected case is $\Theta(n \log n)$.

Analyzing Loops and Recursion

Single Loop

A loop that runs n times is $O(n)$. If the loop variable halves each time (e.g., `while i > 0: i //= 2`), it is $O(\log n)$.

Nested Loops

Two nested loops each running n times yield $O(n^2)$. A loop of n iterations with an inner loop that runs i times yields $O(n^2)$ as well (sum of 1 to n is $n(n+1)/2$).

Recurrence Relations

Recursive algorithms are analyzed using recurrences. For mergesort: $T(n) = 2T(n/2) + O(n)$, which solves to $T(n) = O(n \log n)$ via the Master Theorem.

```
# Fibonacci:  $O(2^n)$  naive vs  $O(n)$  with DP
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

def fib_dp(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Complexity of Sorting Algorithms

Algorithm	Best	Average	Worst	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$

Complexity of Searching Algorithms

Algorithm	Time Complexity	Space
Linear Search	$O(n)$	$O(1)$
Binary Search	$O(\log n)$	$O(1)$
BFS / DFS	$O(V + E)$	$O(V)$
Dijkstra	$O((V + E) \log V)$	$O(V)$
Bellman-Ford	$O(V * E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$

Space Complexity

Space complexity measures the amount of extra memory an algorithm uses. An in-place algorithm uses $O(1)$ extra space (e.g., Bubble Sort). Algorithms that allocate new arrays (e.g., Merge Sort) use $O(n)$ extra space. Recursion adds call

stack space — a recursive function that calls itself n times deep uses $O(n)$ stack space.

Practice Task

For each of the following code snippets, determine the time complexity using Big-O:

```
# Snippet 1
for i in range(n):
    for j in range(i, n):
        print(i, j)

# Snippet 2
i = n
while i > 0:
    print(i)
    i //= 2

# Snippet 3
for i in range(n):
    for j in range(10):
        print(i, j)

# Snippet 4
def mystery(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = mystery(arr[:mid])
    right = mystery(arr[mid:])
    return merge(left, right)
```

Answers: $O(n^2)$, $O(\log n)$, $O(n)$, $O(n \log n)$. Can you explain why?

Dynamic Programming

Dynamic Programming (DP) is an optimization technique for problems with overlapping subproblems and optimal substructure. Instead of solving the same subproblem repeatedly, DP stores results in a table (memoization or tabulation) and reuses them.

The two key properties for DP: (1) **Optimal substructure** — the optimal solution to a problem contains optimal solutions to its subproblems. (2) **Overlapping subproblems** — the same subproblems are solved many times.

 Dynamic programming concept

Memoization (Top-Down)

Memoization is recursion with caching. We write a recursive function that solves the problem, but before computing a result we check a cache (usually a dictionary or array). If the result is already cached, return it; otherwise compute and store it.

```
def fib_memo(n, memo={}):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)  
    return memo[n]
```

Top-down DP is often easier to write because it follows the natural recursive formulation. However, it carries the overhead of recursive calls and can hit recursion depth limits for large inputs.

Tabulation (Bottom-Up)

Tabulation solves the problem iteratively by filling a table from the smallest subproblems upward. This avoids recursion entirely and is usually more efficient in both time and space.

```
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

# Space-optimized O(1) version
def fib_opt(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Classic DP Problems

1. 0/1 Knapsack

Given weights and values of n items, and a capacity W , choose items to maximize total value without exceeding capacity. Each item can be taken at most once.

```
def knapsack_01(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(W + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w], values[i - 1] + dp[i - 1][w - weights[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][W]
```

Time: $O(nW)$, Space: $O(nW)$ or $O(W)$ with 1D optimization.

2. Longest Common Subsequence (LCS)

Given two strings X and Y, find the length of the longest subsequence common to both. Subsequences preserve order but not necessarily contiguity.

```
def lcs(X, Y):
    m, n = len(X), len(Y)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if X[i - 1] == Y[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Time: $O(mn)$, Space: $O(mn)$ or $O(\min(m, n))$ with space optimization.

3. Longest Increasing Subsequence (LIS)

Find the length of the longest subsequence of a given array where elements are strictly increasing.

```

def lis(arr):
    n = len(arr)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# O(n log n) using patience sorting
def lis_optimized(arr):
    import bisect
    tails = []
    for x in arr:
        i = bisect.bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)

```

4. Edit Distance (Levenshtein Distance)

Compute the minimum number of operations (insert, delete, replace) needed to transform one string into another.

```
def edit_distance(A, B):
    m, n = len(A), len(B)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if A[i - 1] == B[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],      # delete
                                   dp[i][j - 1],      # insert
                                   dp[i - 1][j - 1])    # replace
    return dp[m][n]
```

Time: $O(mn)$, Space: $O(mn)$ or $O(n)$.

DP vs Recursion vs Greedy

Feature	Recursion	Dynamic Programming	Greedy
Approach	Divide & conquer	Memoize / tabulate	Local optimal choice
Subproblems	Independent	Overlapping	Single pass
Optimality	Always	Always	Sometimes
Example	Mergesort	Knapsack, LCS	Huffman, Dijkstra

Practice Task

Solve the following DP problems:

1. Given coin denominations [1, 5, 10, 25] and an amount N , find the minimum number of coins needed to make N cents (coin change problem).
2. Given a 2D grid where each cell contains a non-negative number, find a path from top-left to bottom-right that minimizes the sum of numbers (you can only move right or down).

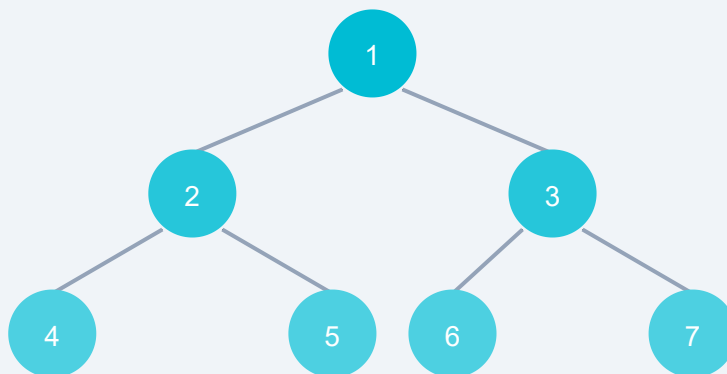
Write both top-down and bottom-up solutions for each.

Greedy Algorithms

A **greedy algorithm** builds a solution step by step by making the locally optimal choice at each step. It never reconsiders its decisions. When a problem has **greedy choice property** and **optimal substructure**, this simple strategy yields the globally optimal solution.

Greedy algorithms are fast (often $O(n \log n)$ or $O(n)$) but only work for problems where local optimal choices lead to a globally optimal solution. This is not always true — which is why we have Dynamic Programming as a fallback.

Binary Tree Traversal



Greedy Choice Property

The greedy choice property states that a globally optimal solution can be arrived at by making a locally optimal (greedy) choice. In other words, you never need to reconsider a choice once it is made.

Optimal Substructure

Optimal substructure means the optimal solution to the overall problem contains optimal solutions to its subproblems. This property is shared with Dynamic Programming, but greedy algorithms use it differently — they solve one subproblem after another without exploring alternatives.

Huffman Coding

Huffman coding is a greedy algorithm for lossless data compression. It builds a variable-length prefix-free code where more frequent characters get shorter codes and less frequent characters get longer codes.

Algorithm

1. Count the frequency of each character in the input.
2. Create a leaf node for each character and add it to a min-heap (priority queue).
3. While more than one node remains:
 - Extract the two nodes with the smallest frequencies.
 - Create a new internal node with these two as children and frequency equal to their sum.
 - Insert the new node back into the heap.
4. The remaining node is the root of the Huffman tree.
5. Assign codes by traversing the tree (left = 0, right = 1).

```

import heapq
from collections import Counter

def huffman_encode(data):
    freq = Counter(data)
    heap = [[w, [char, ""]] for char, w in freq.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        lo = heapq.heappop(heap)
        hi = heapq.heappop(heap)
        for pair in lo[1:]:
            pair[1] = "0" + pair[1]
        for pair in hi[1:]:
            pair[1] = "1" + pair[1]
        heapq.heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])

    return sorted(heap[0][1:], key=lambda p: len(p[1]))

# Usage: huffman_encode("abracadabra")

```

Activity Selection

Given a set of activities with start and end times, select the maximum number of non-overlapping activities. The greedy strategy: always pick the activity with the earliest finish time.

```

def activity_selection(activities):
    # activities: list of (start, end)
    activities.sort(key=lambda x: x[1]) # sort by finish time
    selected = [activities[0]]
    last_end = activities[0][1]
    for start, end in activities[1:]:
        if start >= last_end:
            selected.append((start, end))
            last_end = end
    return selected

```

Time complexity: $O(n \log n)$ due to sorting.

Fractional Knapsack

Unlike the 0/1 Knapsack (which requires DP), the **Fractional Knapsack** allows taking fractions of items. The greedy strategy is to take as much as possible of the item with the highest value-to-weight ratio.

```
def fractional_knapsack(items, capacity):
    # items: list of (value, weight)
    items.sort(key=lambda x: x[0] / x[1], reverse=True)
    total_value = 0.0
    for value, weight in items:
        if capacity >= weight:
            total_value += value
            capacity -= weight
        else:
            total_value += value * (capacity / weight)
            break
    return total_value
```

Time complexity: $O(n \log n)$. This is the classic "take as much as you can of the most valuable thing."

The Fractional Knapsack problem has a greedy solution because you can take fractional amounts. The 0/1 Knapsack (no fractions) requires Dynamic Programming — this subtle difference changes the problem completely.

Traveling Salesman Problem (Approximation)

The Traveling Salesman Problem (TSP) asks: given cities and distances, find the shortest route that visits each city exactly once and returns to the start. TSP is NP-hard, but a simple greedy approximation is the **Nearest Neighbor** heuristic: start at a city and always visit the nearest unvisited city.

```
def tsp_nearest_neighbor(graph, start=0):
    n = len(graph)
    visited = [False] * n
    path = [start]
    visited[start] = True
    current = start
    for _ in range(n - 1):
        next_city = min(
            (i for i in range(n) if not visited[i]),
            key=lambda i: graph[current][i]
        )
        visited[next_city] = True
        path.append(next_city)
        current = next_city
    return path
```

The nearest neighbor heuristic gives a solution within a factor of $O(\log n)$ of optimal in the worst case, but often performs much better in practice.

When Greedy Works vs When It Doesn't

Works (Greedy Choice Property holds)	Does Not Work (Use DP instead)
Huffman Coding	0/1 Knapsack
Activity Selection	Longest Common Subsequence
Fractional Knapsack	Coin Change (some coin systems)
Dijkstra's Shortest Path	Edit Distance
Prim's / Kruskal's MST	Traveling Salesman (optimal)

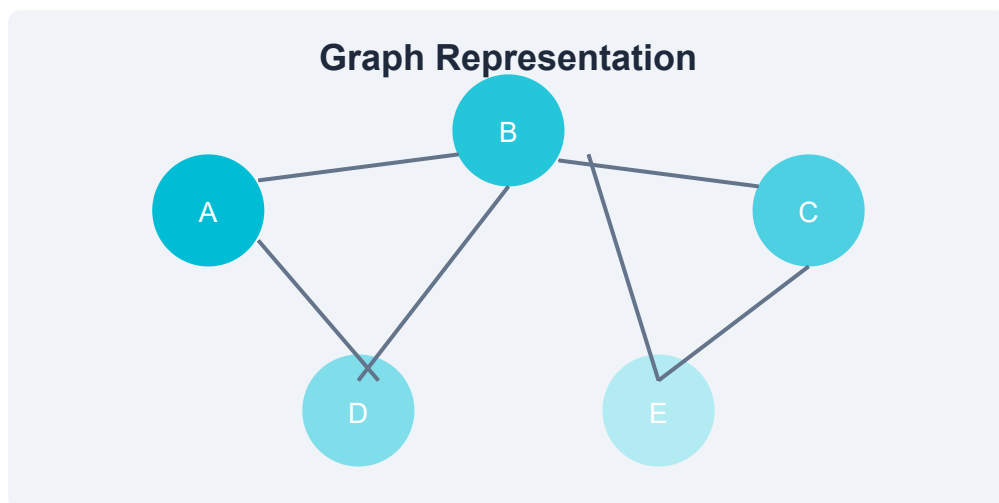
Practice Task

Implement a greedy algorithm to solve the **Minimum Coins** problem: given coin denominations [1, 5, 10, 25] and an amount N , use the greedy strategy (always take the largest possible coin) to return the minimum number of coins. Does this always give the optimal answer? Try with denominations [1, 3, 4] and amount 6 — the greedy answer is 3 ($4+1+1$), but the optimal is 2 ($3+3$).

Practice & Certification

This chapter brings everything together. You will find example problems with solutions, exercises organized by difficulty, quiz questions, a syllabus outline, an 8-week study plan, interview preparation tips, and DSA certification paths.

The best way to learn DSA is to write code every day. Aim to solve at least one problem daily. Start with easy problems, then move to medium, then hard.



Example Problems with Solutions

Problem 1: Two Sum (Easy)

Given an array of integers and a target, return indices of two numbers that add up to the target.

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Time: $O(n)$, Space: $O(n)$.

Problem 2: Maximum Subarray (Medium)

Find the contiguous subarray with the largest sum (Kadane's Algorithm).

```
def max_subarray(nums):
    max_ending = max_sofar = nums[0]
    for x in nums[1:]:
        max_ending = max(x, max_ending + x)
        max_sofar = max(max_sofar, max_ending)
    return max_sofar
```

Time: $O(n)$, Space: $O(1)$.

Problem 3: Merge K Sorted Lists (Hard)

Merge k sorted linked lists into one sorted list. Use a min-heap.

```

import heapq

def merge_k_lists(lists):
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst[0], i, 0))

    result = []
    while heap:
        val, list_idx, elem_idx = heapq.heappop(heap)
        result.append(val)
        if elem_idx + 1 < len(lists[list_idx]):
            next_val = lists[list_idx][elem_idx + 1]
            heapq.heappush(heap, (next_val, list_idx, elem_idx + 1))
    return result

```

Time: $O(N \log k)$, where N is total elements and k is number of lists.

Exercises by Difficulty

Easy

- Find the maximum element in an array.
- Reverse a linked list iteratively and recursively.
- Check if a string is a palindrome.
- Implement binary search on a sorted array.
- Find the first non-repeating character in a string.
- Implement a stack using two queues.
- Count the number of set bits in an integer.

Medium

- Find the longest substring without repeating characters.
- Clone a graph (BFS / DFS approach).
- Check if a binary tree is a valid BST.
- Find the LCA (Lowest Common Ancestor) of two nodes in a binary tree.
- Implement Dijkstra's shortest path algorithm.
- Flatten a binary tree to a linked list.

- Detect a cycle in a linked list and find the start of the cycle.

Hard

- Maximum path sum in a binary tree.
- Serialize and deserialize a binary tree.
- Median of two sorted arrays in $O(\log(m+n))$ time.
- Trapping rain water (two-pointer approach).
- Word ladder (shortest transformation sequence).
- N-Queens problem (backtracking).
- Design a LRU (Least Recently Used) cache.

Quiz Questions

1. What is the time complexity of binary search? *Answer: $O(\log n)$*
2. Which sorting algorithm has the best worst-case time complexity?
Answer: Merge Sort / Heap Sort ($O(n \log n)$)
3. What data structure is used for DFS? *Answer: Stack (recursive call stack)*
4. What is the space complexity of Merge Sort? *Answer: $O(n)$*
5. When does a greedy algorithm fail? *Answer: When the greedy choice property does not hold — local optima do not lead to global optimum.*
6. What is the key difference between Prim's and Kruskal's algorithms?
Answer: Prim grows vertex-by-vertex; Kruskal grows edge-by-edge using Union-Find.
7. State the Max-Flow Min-Cut theorem. *Answer: The maximum flow value equals the minimum cut capacity.*
8. Explain the difference between memoization and tabulation in DP.
Answer: Memoization is top-down (recursion + cache); tabulation is bottom-up (iterative table filling).

Syllabus Outline

Module	Topics
1. Foundations	Complexity analysis, recursion, problem-solving patterns
2. Arrays & Sorting	Array manipulations, sorting algorithms, searching
3. Linked Lists	Singly, doubly, circular, fast & slow pointer
4. Stacks & Queues	Stack, queue, deque, priority queue, monotonic stack
5. Hash Tables	Hashing, collision resolution, hash map patterns
6. Trees	Binary trees, BST, traversals, AVL, heaps, tries
7. Graphs	Representation, BFS, DFS, cycle detection, topological sort
8. Advanced Graphs	Shortest paths, MST, maximum flow
9. DP & Greedy	Dynamic programming, greedy algorithms
10. Advanced Topics	Backtracking, bit manipulation, string algorithms, segment trees

8-Week Study Plan

Week	Focus	Problems per Day
1	Complexity, Arrays, Sorting, Searching	2
2	Linked Lists, Stacks, Queues	2
3	Hash Tables, Recursion	2
4	Trees, BST, Heaps	2
5	Graphs: BFS, DFS, Topological Sort	2
6	Shortest Paths, MST, Maximum Flow	1-2
7	Dynamic Programming	2
8	Greedy, Backtracking, Review & Mock Interviews	3

Interview Preparation Tips

- **Understand the problem** — Ask clarifying questions. Identify input/output constraints. Discuss edge cases.
- **Brute force first** — Propose a naive solution, then optimize. Interviewers want to see your thought process.
- **Use a whiteboard / paper** — Draw diagrams to visualize data structures and algorithms.
- **Test your code** — Walk through example inputs and edge cases (empty, single element, duplicates).
- **Communicate** — Explain your reasoning, time and space complexity, and trade-offs.
- **Practice with a timer** — Simulate real interview conditions on platforms like LeetCode, HackerRank, or CodeSignal.
- **Review fundamentals** — Know array, string, tree, graph, DP, and hash table patterns by heart.

Certification Paths

- **LeetCode** — Weekly contests, study plans, and interview assessments.
- **HackerRank** — Skill certification tests (Problem Solving, Java, Python).
- **CodeSignal** — General Coding Assessment (GCA) used by companies.
- **Google Tech Dev Guide** — Free learning path with curated resources.
- **Coursera / edX** — Data Structures and Algorithms specializations (UC San Diego, Stanford, Princeton).
- **GitHub Copilot** — Use AI-assisted coding to accelerate practice and learn patterns faster.

Certifications are helpful for building confidence and structuring your learning, but portfolio projects and mock interviews are just as important for landing a job.

Practice Task

Pick one easy, one medium, and one hard problem from the lists above. Solve all three without looking at any reference. Time yourself. If you complete the easy in under 10 minutes, the medium in under 25, and the hard in under 45, you are ready for most technical interviews.

References & Quick Guide

This chapter collects all the essential reference material from the tutorial into one place. Use these tables as a quick cheatsheet when solving problems or preparing for interviews.

Bookmark this page. It compresses 14 chapters of DSA knowledge into a single-page quick guide you can refer to anytime.

 DSA reference sheet

Data Structures — Operations & Complexities

Data Structure	Access	Search	Insert	Delete	Space
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Linked List (Singly)	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Stack (LIFO)	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue (FIFO)	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Deque	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Hash Table	N/A	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Heap (Min/Max)	$O(1)$	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Graph (adjacency list)	$O(1)$	$O(V + E)$	$O(1)$	$O(E)$	$O(V + E)$
Trie (prefix tree)	$O(k)$	$O(k)$	$O(k)$	$O(k)$	$O(n * k)$

* Average case for hash tables. Worst case: $O(n)$ due to collisions.

Sorting Algorithms Reference

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix Sort	$O(d * (n + k))$	$O(d * (n + k))$	$O(d * (n + k))$	$O(n + k)$	Yes

Searching Algorithms Reference

Algorithm	Data Structure	Time Complexity	Space Complexity
Linear Search	Array / List	$O(n)$	$O(1)$
Binary Search	Sorted Array	$O(\log n)$	$O(1)$
BFS (Breadth-First Search)	Graph / Tree	$O(V + E)$	$O(V)$
DFS (Depth-First Search)	Graph / Tree	$O(V + E)$	$O(V)$
Dijkstra (SSSP)	Graph (non-negative weights)	$O((V + E) \log V)$	$O(V)$
Bellman-Ford (SSSP)	Graph (allows negative weights)	$O(V * E)$	$O(V)$
Floyd-Warshall (APSP)	Graph (allows negative weights)	$O(V^3)$	$O(V^2)$
A* Search	Graph with heuristic	$O(E)$ (w/ good heuristic)	$O(V)$
Prim's Algorithm (MST)	Graph (undirected)	$O(E \log V)$	$O(V)$
Kruskal's Algorithm (MST)	Graph (undirected)	$O(E \log E)$	$O(V)$
Ford-Fulkerson (Max Flow)	Flow Network	$O(E * f)$	$O(V)$
Edmonds-Karp (Max Flow)	Flow Network	$O(V * E^2)$	$O(V)$

Graph Algorithms Reference

Problem	Algorithms	Key Idea
Shortest Path (unweighted)	BFS	Queue, $O(V + E)$
Shortest Path (non-negative)	Dijkstra	Priority queue, greedy
Shortest Path (negative edges)	Bellman-Ford	Relax $V-1$ times, detect negative cycles
All-Pairs Shortest Path	Floyd-Warshall	DP over intermediate vertices
Minimum Spanning Tree	Prim / Kruskal	Greedy: vertex vs edge based
Maximum Flow	Ford-Fulkerson / Edmonds-Karp	Augmenting paths, residual graph
Topological Sort	DFS / Kahn's	Ordering DAG vertices
Cycle Detection (directed)	DFS with colors / Kahn's	Back edges or in-degree tracking
Bipartite Check	BFS with colors	2-coloring
Strongly Connected Components	Kosaraju / Tarjan	DFS + transpose / low-link values

Dynamic Programming Patterns

- **Fibonacci-style** — $DP[i] = DP[i-1] + DP[i-2]$. Examples: climbing stairs, Fibonacci.
- **Knapsack-style** — $DP[i][w] = \max(DP[i-1][w], \text{value}[i] + DP[i-1][w - \text{weight}[i]])$. Examples: 0/1 knapsack, subset sum, partition equal sum.

- **Longest Common Subsequence** — $DP[i][j] = DP[i-1][j-1] + 1$ if match, else $\max(DP[i-1][j], DP[i][j-1])$. Examples: LCS, edit distance, longest palindromic subsequence.
- **Longest Increasing Subsequence** — $DP[i] = 1 + \max(DP[j])$ for $j < i$ and $arr[j] < arr[i]$. $O(n^2)$ or $O(n \log n)$ with patience sorting.
- **Interval DP** — $DP[i][j]$ = optimal solution for subarray $i..j$. Examples: matrix chain multiplication, burst balloons.
- **Tree DP** — DP on tree nodes, usually combining child results. Examples: diameter of tree, tree max independent set.

Quick Guide Cheatsheet

Big-O Complexity Classes

$O(1)$	$< O(\log n)$	$< O(n)$	$< O(n \log n)$	$< O(n^2)$	$< O(2^n)$	$< O(n!)$
constant	logarithmic	linear	linearithmic	quadratic	exponential	factorial

Common Recurrence Solutions

$T(n) = 2T(n/2) + O(n)$	$\rightarrow O(n \log n)$	(Merge Sort)
$T(n) = T(n-1) + O(1)$	$\rightarrow O(n)$	(Linear recursion)
$T(n) = T(n-1) + T(n-2)$	$\rightarrow O(2^n)$	(Naive Fibonacci)
$T(n) = 2T(n-1) + O(1)$	$\rightarrow O(2^n)$	(Subset generation)

Python Snippets for Quick Reference

```
# Binary search
lo, hi = 0, len(arr) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if arr[mid] == target: break
    elif arr[mid] < target: lo = mid + 1
    else: hi = mid - 1

# Tree traversals
def inorder(node): inorder(node.left); print(node); inorder(node.right)
def preorder(node): print(node); preorder(node.left); preorder(node.right)
def postorder(node): postorder(node.left); postorder(node.right); print(node)

# BFS
queue = [start]; visited = {start}
while queue:
    u = queue.pop(0)
    for v in graph[u]:
        if v not in visited: visited.add(v); queue.append(v)

# DFS recursion
def dfs(u):
    visited.add(u)
    for v in graph[u]:
        if v not in visited: dfs(v)

# DSU find with path compression
def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]
```

This reference sheet covers roughly 90% of what you will encounter in a DSA coding interview. Master these patterns and you will be well prepared.

Practice Task

Pick any three algorithms from the reference tables above that you feel least confident about. Implement them from scratch without looking at any reference. Then compare your implementation to the versions in the earlier chapters. Repeat until you can implement all of them from memory.